

DESC

HISTORY

RULES

README

SETTINGS

MORE

- For `fish = [1, 4, 3, 2]` and `baits = [1, 1]`, the output should be `solution(fish, baits) = 3`.

Explanation:

While there are two baits available, it is only possible to catch three fish since both baits are not strictly smaller than the first fish, which has a size of `fish[0] = 1`.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.integer fish

An array of integers representing the sizes of fish available in the pond.

main.cpp

Saved

```
1 int solution(vector<int> fish, vector<int> baits) {
2
3 }
4
```

TESTS

CUSTOM

Search

DESC

HISTORY

RULES

README

SETTINGS

• [Input] array.integer baits

An array of integers representing the sizes of baits you have available.

Guaranteed constraints:

$$1 \leq \text{baits.length} \leq 100,$$

$$1 \leq \text{baits}[i] \leq 10^9.$$

• [output] integer

The maximum number of fish you can catch from the pond.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloworld(string name) {
    cout << "This prints to the console" << endl;
    return "Hello, " + name;
}
```

@OAHELPCODER

main.cpp

Saved

```
1 int solution(vector<int> baits, int fish) {
2     // Write your code here
3 }
4
```

TESTS

CUSTOM

MORE



Search



144Hz

F1

F2

F3

DESC

HISTORY

RULES

README

SETTINGS

MORE

• [memory limit] 1 GB

• [input] array.integer fish

An array of integers representing the sizes of fish available in the pond.

Guaranteed constraints:

$1 \leq \text{fish.length} \leq 100$,

$1 \leq \text{fish}[i] \leq 10^9$.

• [input] array.integer baits

An array of integers representing the sizes of baits you have available.

Guaranteed constraints:

$1 \leq \text{baits.length} \leq 100$,

$1 \leq \text{baits}[i] \leq 10^9$.

• [output] integer

The maximum number of fish you can catch from the pond.

main.cpp

Saved

```
1 int solution(va
2
3 }
4
```

TESTS

CUSTOM

Search

DESC

HISTORY

RULES

README

SETTINGS

To compute the answer, you need to use each bait to its possible extent, going from the largest bait to the smallest bait. Use each bait to catch the largest fish remaining in the pond and move to the next bait if the current bait was used three times or if it is not strictly smaller than the largest remaining fish. When you run out of baits, return the number of caught fish.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\max(\text{fish.length}, \text{baits.length})^3)$ will fit within the execution time limit.

Example

- For `fish = [1, 2, 3]` and `baits = [1]`, the output should be `solution(fish, baits) = 2`.

Explanation:

main.cpp

Saved

```
1 int solution(vs
2
3 }
4
```

TESTS

CUSTOM

MORE



Search



7440



DESC



HISTORY



RULES



README



SETTINGS

⋮ MORE

- For `fish = [1, 2, 3]` and `baits = [1]`, the output should be `solution(fish, baits) = 2`.

Explanation:

Using `baits[0] = 1`, it is possible to catch 2 fish `fish[2] = 3` and `fish[1] = 2`, since $1 < 3$ and $1 < 2$. The remaining fish `fish[0] = 1` can't be caught using the given bait.

- For `fish = [2, 2, 3, 4]` and `baits = [1]`, the output should be `solution(fish, baits) = 3`.

Explanation:

Since there is only one bait, it is possible to catch only three fish (any of the given four since the bait has a smaller size than any of the fish) because the bait can only be used three times.

main.cpp

Saved

```
1 int solution(vector<int> fish, int baits) {
2     // Write your code here
3 }
4
```

TESTS

CUSTOM



Search



144Hz

F1

F2

F3

F4



DESC



HISTORY



RULES



README



SETTINGS

Codewriting



Imagine that you are going fishing at the local pond. The size of the bait must be strictly smaller than the size of the fish for it to catch. Once the fish is caught, it is removed from the pond and cannot be caught again. However, each bait can be used up to 3 times before depletion.

Given two arrays of `fish` and `baits`, where `fish[i]` corresponds to the size of the i^{th} fish in the pond, and `baits[j]` corresponds to the size of the j^{th} bait, your task is to return the maximum number of fish you can catch from the pond with the given baits.

To compute the answer, you need to use each bait to its possible extent, going from the largest bait to the smallest bait. Use each bait to catch the largest fish remaining in the pond.

main.cpp

Saved

```
1 int solution(  
2  
3 }  
4
```

MORE



Search



DESC



HISTORY



RULES



README



SETTINGS

Codewriting

You are given a string of digits `panel` and an array of strings `codes`. Each string in the `codes` array consists of digits only and represents a code in the following format: "`<index><pattern>`", where both `index` and `pattern` should consist of at least one digit. Since there are several ways to split the code, let's consider them all in ascending order of `index` length and call them split-cases. For instance, for the `code = "1324"`, the split-cases are:

- split-case 1: `index = "1"` and `pattern = "324"` ;
- split-case 2: `index = "13"` and `pattern = "24"` ;
- split-case 3: `index = "132"` and `pattern = "4"` .

For each code in `codes` and for every split-case of this code, check whether a string `pattern` is present at the `index` position in the `panel` string. Return a string array consisting of results of these checks, where each element is either `pattern`, if this `pattern` is present in `panel`, or otherwise `"not found"`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than

$O(\text{codes.length} \times \max(\text{codes}) \times \text{panel.length})$ will fit within the execution time limit.

Example

: MORE

TESTS



DESC



HISTORY



RULES



README



SETTINGS

: MORE

- "39"
- index is 63 and pattern is "9". As `panel[63]` doesn't exist, this is not a valid code. The answer for this split-case is "not found".
- The resulting array is `["not found", "11", "not found", "39", "not found"]`.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB

- [input] string panel

A string of digits.

Guaranteed constraints:

$$1 \leq \text{panel.length} \leq 10^3$$

- [input] array.string codes

An array of strings of digits.

Guaranteed constraints:

$$1 \leq \text{codes.length} \leq 100$$

$$2 \leq \text{codes}[i].\text{length} \leq 10$$

- [output] array.string

An array of strings, where each string is either the existing

main.cpp

```
1 vector<stri
2
3 }
4
```

TESTS

CUSTO

esc

F1

F2

F3

F4

F5



DESC



HISTORY



RULES



README



SETTINGS

Example

- For `panel = "2311453915"` and `codes = ["0211", "639"]`, the output should be `solution(panel, codes) = ["not found", "11", "not found", "39", "not found"]`.

Explanation:

Let's consider all possible codes:

- For `codes[0] = "0211"` there are three split-cases:
 - `index` is 0 and `pattern` is "211". Since `panel[0..2] = "231" ≠ pattern`, this is not a valid code. The answer for this split-case is "not found".
 - `index` is 02 and `pattern` is "11". `panel[2..3] = "11" = pattern`, so this is a valid code. The answer for this split-case is "11".
 - `index` is 021 and `pattern` is "1". As `panel[21]` doesn't exist, this is not a valid code. The answer for this split-case is "not found".
- For `codes[1] = "639"` there are two split-cases:
 - `index` is 6 and `pattern` is "39". `panel[6..7] = "39" = pattern`, so this is a valid code. The answer for this split-case is "39".

main.cpp

```
1 vector<stri
2
3 }
4
```

: MORE

TESTS

CUST



memory limit] 1 GB

] string panel

of digits.

eed constraints:

el.length $\leq 10^3$.

rray.string codes

if strings of digits.

d constraints:

.length ≤ 100 ,

ij.length ≤ 10 .

ny.string

strings, where each string is either the existing

"not found".

essage to the console

ig

string name) {

prints to the console when you Run Te

" + name;

TESTS

CUSTOM TESTS

0/300

VIEW

MacBook Air

DESC

HISTORY

RULES

README

SETTINGS

- [memory limit] 1 GB

- [input] string panel

A string of digits.

Guaranteed constraints:

$$1 \leq \text{panel.length} \leq 10^3.$$

- [input] array.string codes

An array of strings of digits.

Guaranteed constraints:

$$1 \leq \text{codes.length} \leq 100,$$

$$2 \leq \text{codes}[i].\text{length} \leq 10.$$

- [output] array.string

An array of strings, where each string is either the existing

pattern Or "not found".

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Te
    return "Hello, " + name;
}
```

```
1 vector<string>
2
3 }
4
```

TESTS

CUSTOM

⋮ MORE

Mac

esc

F1

F2

F3

F4

F5

~

!

@

#

\$

%

Given a board of cells containing a bubble of a specific color, your task is to emulate a bubble-popping game. In this game, the player can click a cell every turn to pop bubbles. After clicking the cell, the following happens:

- The bubble in the clicked cell and bubbles of the same color among its diagonal neighbors are "popped" and removed, resulting in empty cells.
- After bubbles are removed, the remaining bubbles in cells above the empty cells drop down to fill all empty cells.
- Nothing happens if the clicked cell is empty (it does not contain a bubble).

The input consists of two arrays:

- An initial board of cells `bubbles` - a multidimensional array of integers representing cells containing different colored bubbles;
- A set of player turns `operations` - 2-element integer arrays describing the coordinates (the row and column) of the cell that the player clicked on during each turn.

Return the state of the game board after all `operations` are processed. The output should be a multidimensional array of integers with the same size as `bubbles`, but replace the integers in all empty cells (without bubbles) with `0`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than

Return the state of the game board after all `operations` are processed. The output should be a multidimensional array of integers with the same size as `bubbles`, but replace the integers in all empty cells (without bubbles) with `0`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than

$O(\text{bubbles.length}^2 \cdot \text{bubbles}[0].\text{length}^2 \cdot \text{operations.length})$ will fit within the execution time limit.

Example

For

```
bubbles = [[1, 1, 1, 4, 3],
            [4, 1, 2, 3, 3],
            [1, 5, 1, 1, 2],
            [4, 3, 2, 2, 4]]
```

and

```
operations = [[1, 1],
              [3, 3],
              [2, 2],
              [3, 0]]
```

the output should be

```
solution(bubbles, operations) = [[0, 0, 0, 0, 0],
```



K TO QUESTIONS



4: (3, 0)

Note: If you cannot view the video properly, please use [this link](#) to access it.

Input/Output

- [execution time limit] 4 seconds (py3)
- [memory limit] 1 GB
- [input] array.array.integer bubbles

A multidimensional array of integers representing the initial game board of cells containing bubbles. Integers represent the colors of bubbles within cells, and it is guaranteed that all cells will have bubbles initially.

Guaranteed constraints:

$$1 \leq \text{bubbles.length} \leq 30,$$
$$1 \leq \text{bubbles}[0].\text{length} \leq 30,$$
$$1 \leq \text{bubbles}[i][j] \leq 900.$$

- [input] array.array.integer operations

An array of 2-element arrays representing the player's moves - integers represent the coordinates of the row and column that the player has clicked on each turn.

Guaranteed constraints:

K TO QUESTIONS

```
operations = [[1, 1],  
              [3, 3],  
              [2, 2],  
              [3, 0]]
```

the output should be

```
solution(bubbles, operations) = [[0, 0, 0, 0, 0],  
                                [0, 1, 0, 4, 3],  
                                [0, 5, 0, 3, 3],  
                                [4, 3, 2, 1, 4]]
```

▼ Expand to see the example video.



Note: If you cannot view the video properly, please use [this link](#) to access it.

Input/Output

$1 \leq \text{bubbles}[0].\text{length} \leq 30$
 $1 \leq \text{bubbles}[i][j] \leq 900$

• [input] array.array.integer operations

An array of 2-element arrays representing the player's moves - integers represent the coordinates of the row and column that the player has clicked on each turn.

Guaranteed constraints:

 $1 \leq \text{operations}.\text{length} \leq 200$
 $\text{operations}[i].\text{length} = 2$
 $0 \leq \text{operations}[i][0] \leq \text{bubbles}.\text{length}$
 $0 \leq \text{operations}[i][1] \leq \text{bubbles}[0].\text{length}$

• [output] array.array.integer

Return the final state of the game board after all operations are processed. The output should be a multidimensional array of integers with the same size as `bubbles`, with 0 representing empty cells without bubbles.

[Python 3] Syntax Tips

```
# Prints help message to the console
# Returns a string
def helloWorld(name):
    print("This prints to the console when you Run Test")
    return "Hello, " + name
```


Your task is to simulate how the figure should fall, and find the minimum number of obstacles that should be removed to let the figure finally touch the bottom of the board with at least one of its cells.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{board.length} \cdot \text{board}[0].\text{length})$ will fit within the execution time limit.

Example

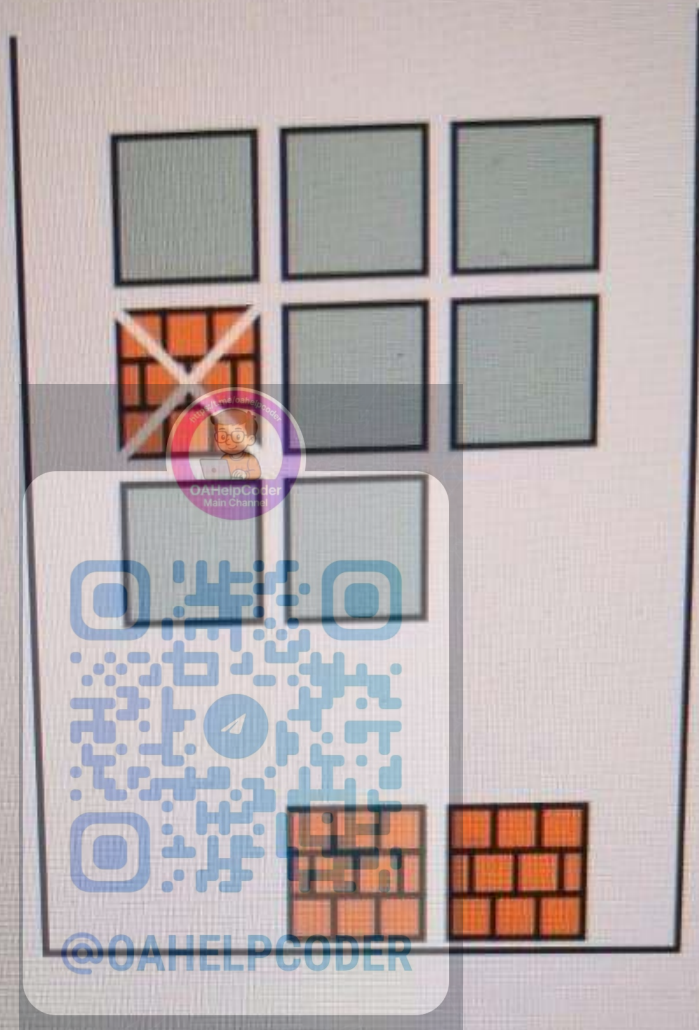
- For

```
board = [['*', '*', '*'],
         ['#', '*', '*'],
         ['*', '*', '-'],
         ['- ', '- ', '-'],
         ['- ', '#', '#']]
```

the output should be `solution(board) = 2`.

▼ Expand to see the example video.

▼ Expand to see the example



Note: If you are unable to view the video properly, please use [this link](#) to access it.

There were 2 obstacles that needed to be cleared for the figure to reach the bottom, so the answer is 2.

⋮ MORE

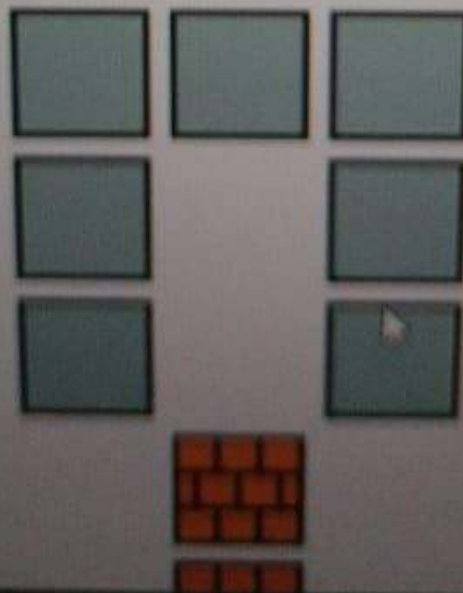
There were 2 obstacles that needed to be cleared for the figure to reach the bottom, so the answer is 2 .

- For

```
board = [[ '*', '*', '*', '*', '*', '*'],  
          ['*', '*', '-', '*', '*'],  
          ['*', '*', '*', '*', '*', '*'],  
          ['-', '#', '*', '*', '*', '*'],  
          ['-', '#', '*', '*', '*', '*']]
```

the output should be `solution(board) = 0` .

▼ Expand to see the example video.



Note: If you are unable to view the video properly, please use [this link](#) to access it.

The figure can reach the bottom without removing any obstacles, so the answer is 0.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.array.char board

A rectangular matrix representing the board consisting of the characters `'-'`, `'*'`, and `'#'`. It's guaranteed that one of the cells of the board will be `'*'` (part of the figure).

Guaranteed constraints:

$2 \leq \text{board.length} \leq 100$

[input] array.array.char board

A rectangular matrix representing the board consisting of the characters '-', '*', and '#'. It's guaranteed that one of the cells of the board will be '*' (part of the figure).

Guaranteed constraints:

$2 \leq \text{board.length} \leq 100$

$2 \leq \text{board}[i].\text{length} \leq 100$

- [output] integer

The minimum number of obstacles that should be removed to make the figure reach the bottom.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when
    return "Hello, " + name;
}
```



@OAHELPCODER

Access Needs Visa's Help

CodeSignal

app.codesignal.com

BACK TO QUESTIONS

DESC

HISTORY

RULES

README

SETTINGS

MORE

1 Codewriting

You are moderating a newspaper page, and you have to align the text on the page properly. The text is provided to you in the following format:

- `paragraphs` is an array of paragraphs, where each paragraph is represented as an array of words.
- `aligns` is an array representing the alignment of each paragraph from `paragraphs` - each element is either `"LEFT"` or `"RIGHT"`.
- `width` represents the maximum number of characters each line of the output can include.

Your task is to produce a newspaper page according to the following specifications:

- For each paragraph `paragraphs[i]`

main.cpp

```
<pre>#include <iostream>
using namespace std;
vector<vector<string>>> paragraphs;
vector<string> aligns;
int width;

// Function to produce the newspaper page
// Returns a string representing the newspaper page
string produceNewspaperPage() {
    // Your code here
    return "";
}

int main() {
    // Read input
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) {
        int m;
        cin >> m;
        vector<string> p(m);
        for (int j = 0; j < m; j++) {
            cin >> p[j];
        }
        paragraphs.push_back(p);
    }
    int k;
    cin >> k;
    vector<string> a(k);
    for (int i = 0; i < k; i++) {
        cin >> a[i];
    }
    aligns = a;
    int l;
    cin >> l;
    width = l;

    // Produce the newspaper page
    string result = produceNewspaperPage();

    // Print the result
    cout << result << endl;

    return 0;
}</pre>
```

TESTS

CUSTOM TESTS

RESULTS

Test 2

Test 3

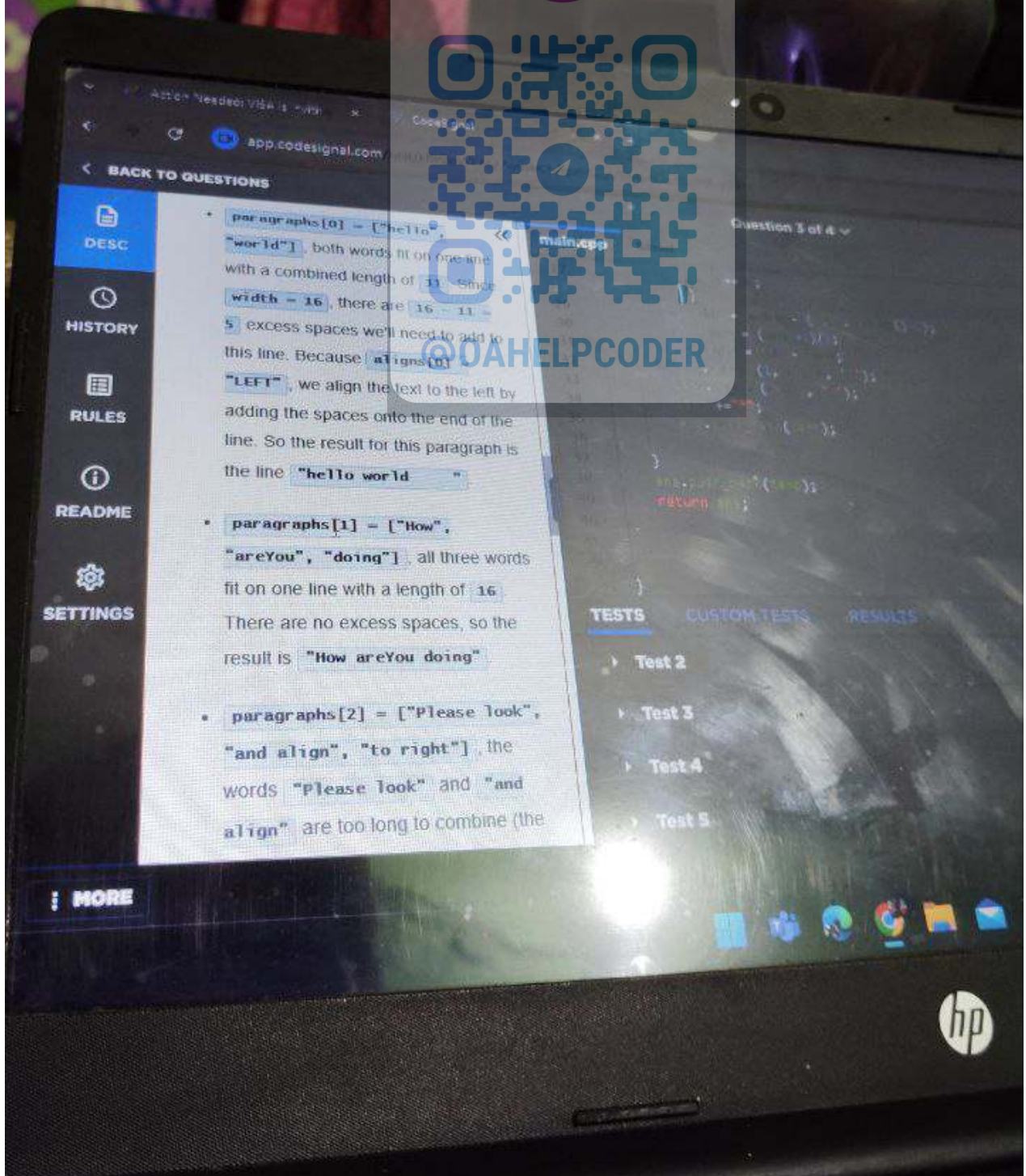
Test 4

Test 5





@OAHELPCODER



BACK TO QUESTIONS

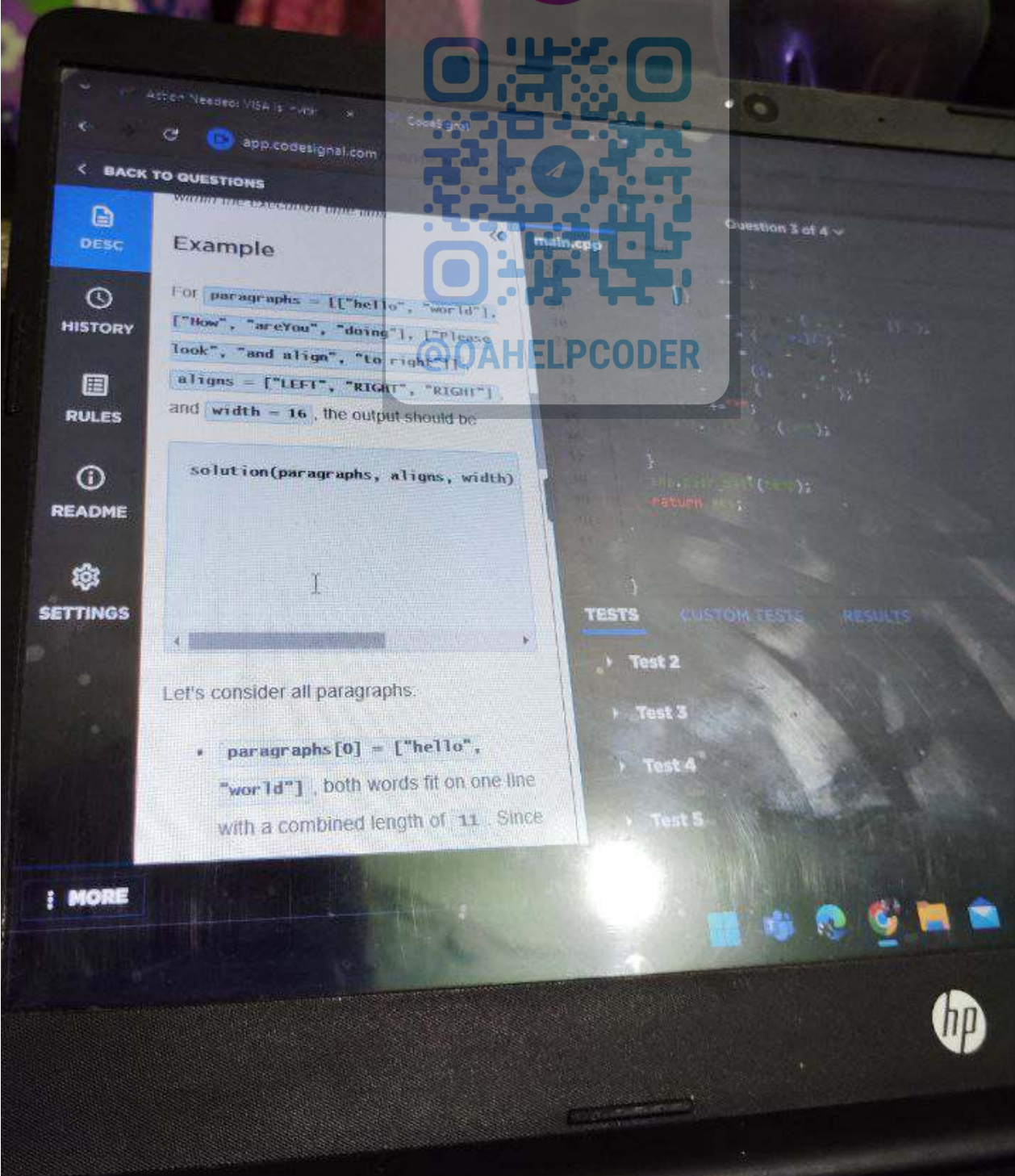
- `paragraphs[0] = ["hello", "world"]`, both words fit on one line with a combined length of `11`. Since `width = 16`, there are `16 - 11 = 5` excess spaces we'll need to add to this line. Because `aligns[0] = "LEFT"`, we align the text to the left by adding the spaces onto the end of the line. So the result for this paragraph is the line `"hello world "`.
- `paragraphs[1] = ["How", "areYou", "doing"]`, all three words fit on one line with a length of `16`. There are no excess spaces, so the result is `"How areYou doing"`.
- `paragraphs[2] = ["Please look", "and align", "to right"]`, the words `"Please look"` and `"and align"` are too long to combine (the

TESTS CUSTOM TESTS RESULTS

- ▶ Test 2
- ▶ Test 3
- ▶ Test 4
- ▶ Test 5

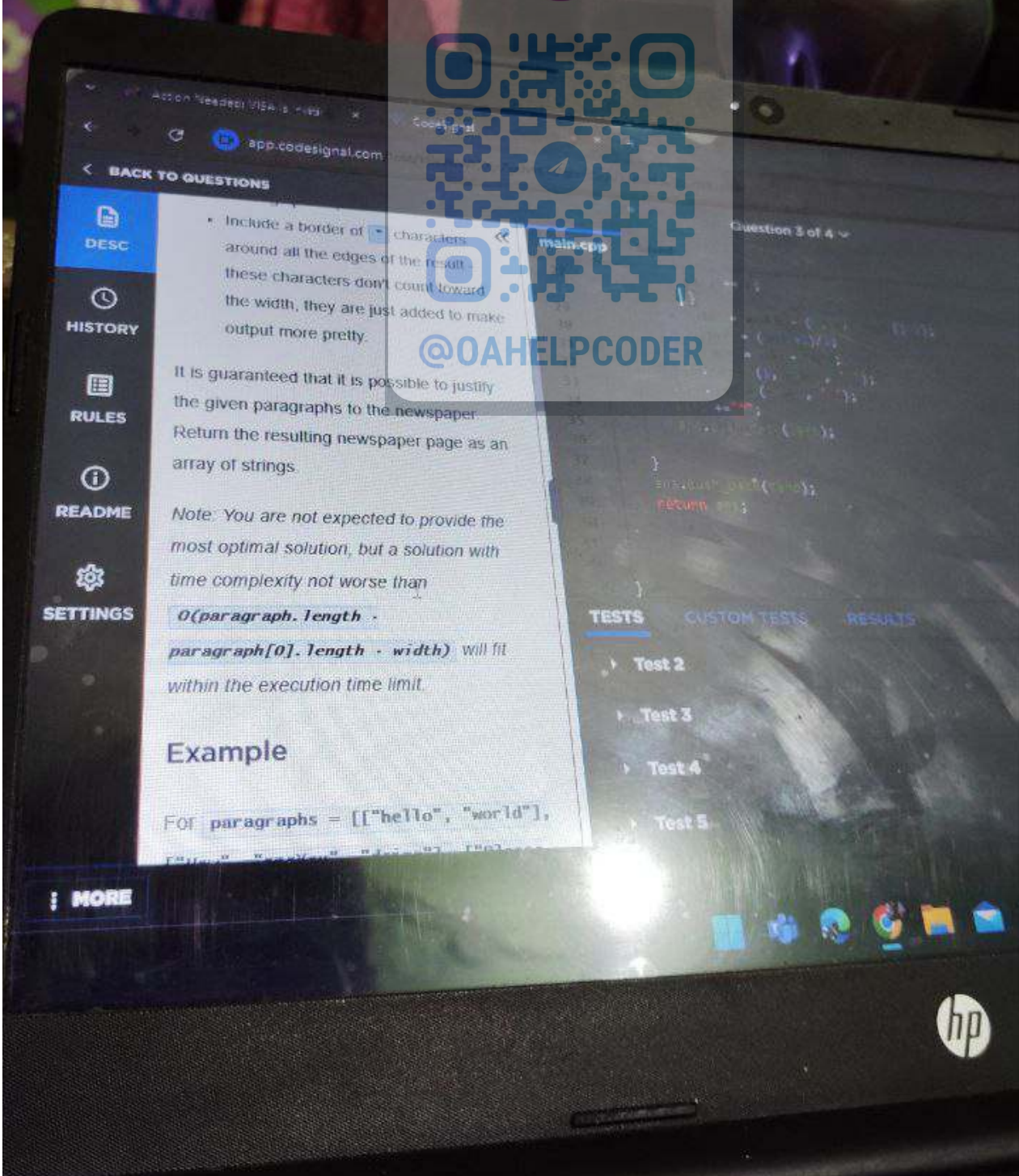


@OAHELPCODER



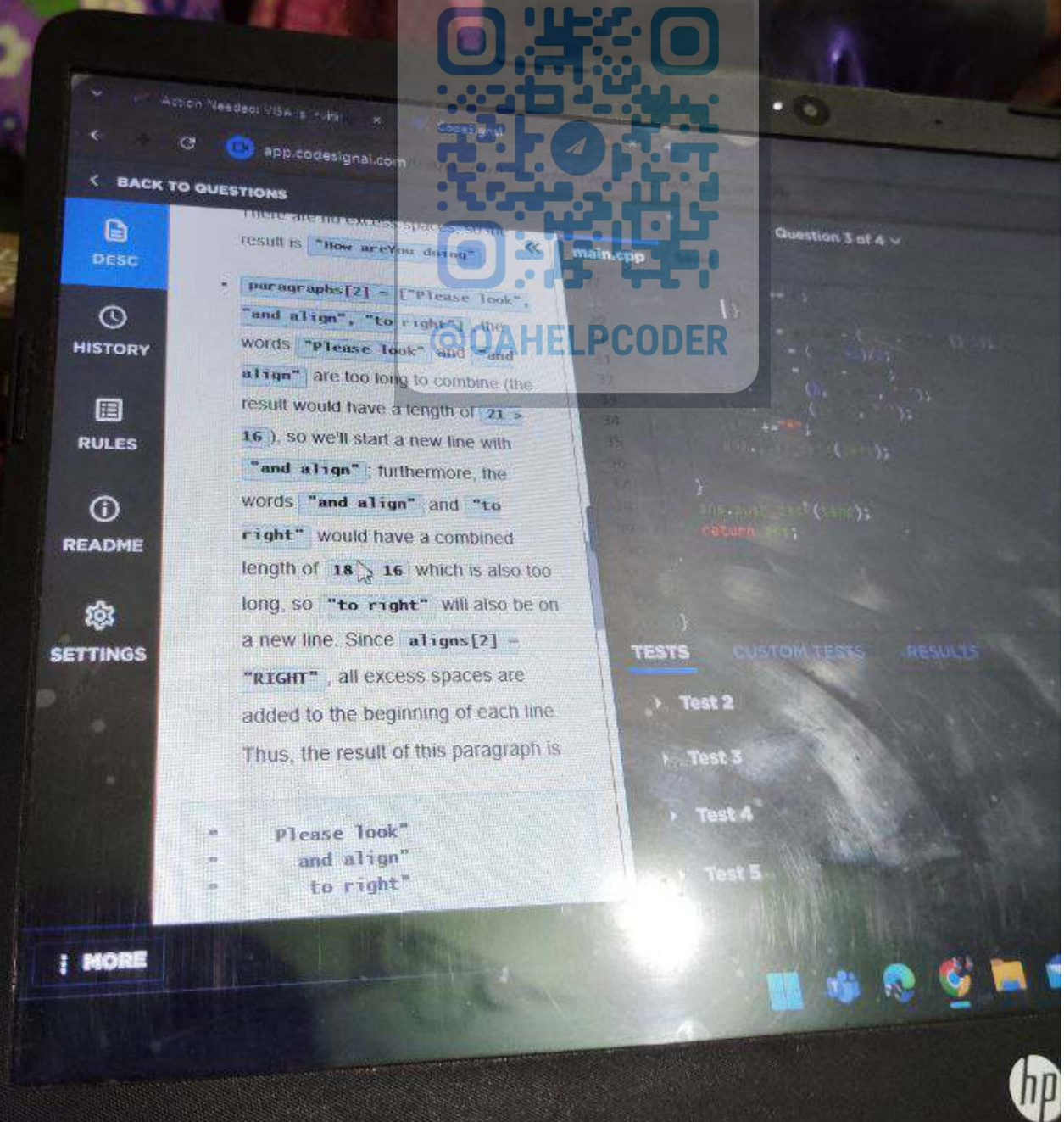


@OAHELPCODER



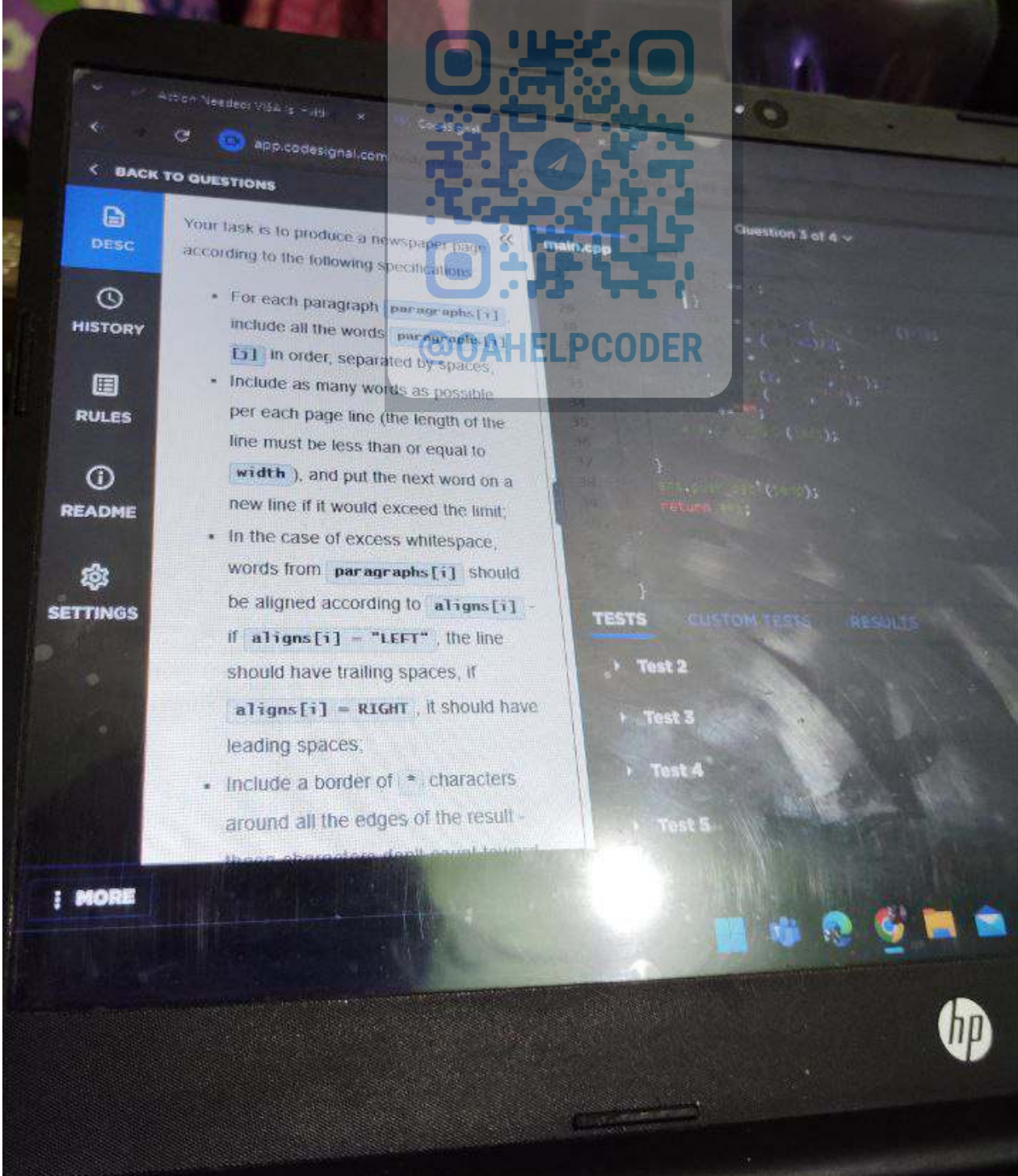


@OAHELPCODER





@OAHELPCODER



Your task is to produce a newspaper page according to the following specifications:

- For each paragraph `paragraphs[i]` include all the words `paragraphs[i]` in order, separated by spaces.
- Include as many words as possible per each page line (the length of the line must be less than or equal to `width`), and put the next word on a new line if it would exceed the limit.
- In the case of excess whitespace, words from `paragraphs[i]` should be aligned according to `aligns[i]` - if `aligns[i] = "LEFT"`, the line should have trailing spaces, if `aligns[i] = "RIGHT"`, it should have leading spaces.
- Include a border of `*` characters around all the edges of the result - these characters don't count towards

TESTS CUSTOM TESTS RESULTS

Test 2
Test 3
Test 4
Test 5

❶ Codewriting

You are developing a feature for a linguistics research tool aimed at analyzing vowel patterns within words. Linguists are particularly interested in sequences of vowels ("a", "e", "i", "o", or "u") within the text.

Given a string `text` consisting of lowercase English letters, count the number of **substrings** of length 3 that contain exactly two vowels (not necessarily different ones).

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{text.length}^2)$ will fit within the execution time limit.

Example

- For `text = "aeiobe"`, the output should be `solution(text) = 2`.

Explanation:

There are two substrings of length 3 with exactly two vowels:

- `"iob"` — 2 vowels: `'i'` and `'o'`.
- `"obe"` — 2 vowels: `'o'` and `'e'`.

- For `text = ""`, the output should be `solution(text) = 0`.

- "nan" — 1 vowel.
- "ana" — 2 vowels: 'a' and 'a'.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] string text

A string consisting of lowercase English letters.

Guaranteed constraints:

$0 \leq \text{text.length} \leq 1000$

- [output] integer

The number of substrings of length 3 in the given string that contain exactly two vowels.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Te
    return "Hello, " + name;
}
```

main.cpp

```
1 int solution(string text) {  
2  
3 }  
4
```



@OAHELPCODER

TESTS

CUSTOM TESTS

MacBook Air

Example



- For `text = "aeiobe"`, the output should be `solution(text) = 2`.

Explanation:

There are two substrings of length 3 with exactly two vowels:

- `"iob"` — 2 vowels: 'i' and 'o'.
- `"obe"` — 2 vowels: 'o' and 'e'.

- For `text = ""`, the output should be `solution(text) = 0`.

Explanation:

The input string is empty, so there are no substrings of length 3.

- For `text = "banana"`, the output should be `solution(text) = 2`.

Explanation:

There are 2 substrings of length 3 with exactly two vowels:

- `"ban"` — 1 vowel.
- `"ana"` — 2 vowels: 'a' and 'a'.
- `"nan"` — 1 vowel.
- `"ana"` — 2 vowels: 'a' and 'a'.

There are 2 substrings of length 3 with exactly two vowels:

- "ban" — 1 vowel.
- "ana" — 2 vowels: 'a' and 'a'.
- "nan" — 1 vowel.
- "ana" — 2 vowels: 'a' and 'a'.

Input/Output

- [execution time limit] 0.5 seconds (cpp)

- [memory limit] 1 GB

- [input] string text

A string consisting of lowercase English letters.

Guaranteed constraints:

$0 \leq \text{text.length} \leq 1000$

- [output] integer

The number of substrings of length 3 in the given string that contain exactly two vowels.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
```


Codewriting

Question 3 of 4

You are given an array `schedules` representing existing meetings for all employees over the course of an entire day, and an integer `length` representing the length of a new meeting in minutes.

Each element in `schedules` is an array, such that `schedules[i][j]` represents the j^{th} meeting for the i^{th} employee. Each meeting is represented by a pair of integers: `[startTime, finishTime]`, where each integer represents the number of minutes passed since the beginning of the day.

`startTime` and `finishTime` do not exceed 24×60 .

Your task is to find the earliest possible time when a meeting of length `length` can be scheduled for all employees. If there is no time block which suits everyone, return `-1`.

Note: The new meeting should also fit within the same day, so the finish time for this meeting should not exceed 24×60 .

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{schedules.length})$.

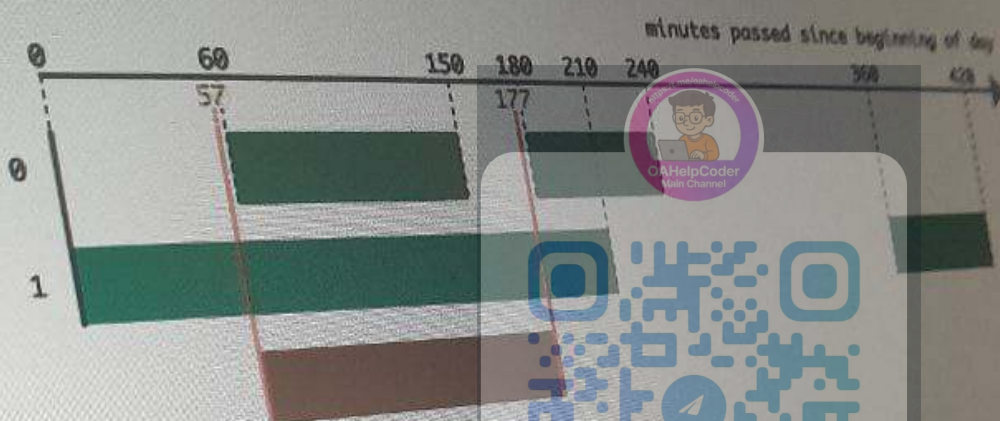
MORE



Search

ASUS

▼ Expand to see the example video.



Note: If you are unable to view the video properly, please use [this link](#) to access it.

Explanation:

If the new meeting is scheduled to start from minute 240, it will last until minute 360. The interval $[240, 360]$ does not coincide with any other intervals from schedules, so it's possible to schedule this meeting to allow all employees to

MORE



Q Search

ASUS





@OAHELPCODER

main.cpp

```
1 solution  
(vector<vector<vector<int>>> schedules,  
  length) {
```

the schedules of all employees
of the day.

return end;

TESTS

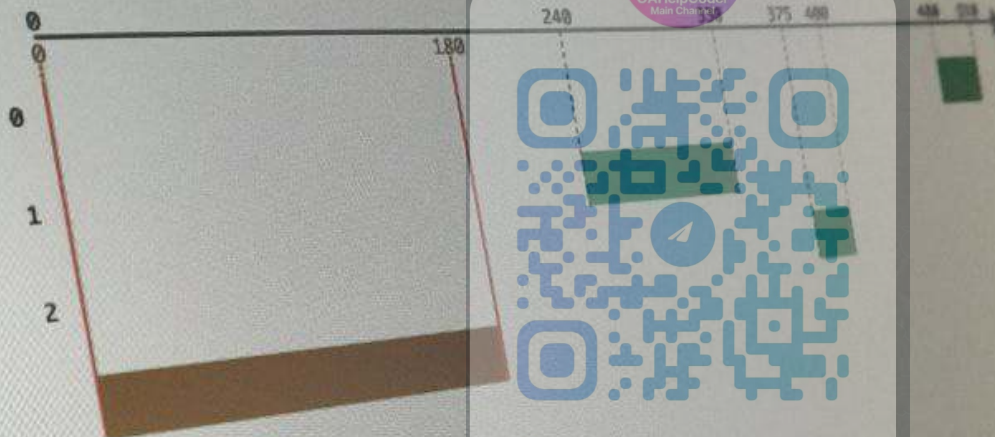
QUESTIONS

1

Question 3 of 4

and `length = 180`, the output should be `solution(schedules, length) =`

0



Explanation:

If the new meeting is scheduled right at the beginning of the day, then it wouldn't conflict with any other meetings, so the answer is 0.

- For

MORE

QUESTIONS

Explanation:

If the new meeting is scheduled right at the beginning of the day, then it wouldn't conflict with any other meetings, so the answer is 0.

- For

```
schedules = [  
    [[0, 1439]],  
    [[0, 1439]],  
    [[0, 390], [480, 510]]  
]
```

and `length = 90`, the output should be `solution(schedules, length) = -1`.

Explanation:

The first two employees are booked for the whole day, so it's not possible to have a meeting with a duration of 90 minutes that all the employees can attend. Thus the answer is -1.

Input/Output



@OAHELPCODER

QUESTIONS

Note: If you are unable to view the video properly, please use [this link](#) to access it.

Explanation:

If the new meeting is scheduled to start from minute 240, it will last until minute 360. The interval [240, 360] does not coincide with any other intervals from `schedules`, so it's possible to schedule this meeting to allow all employees to attend.

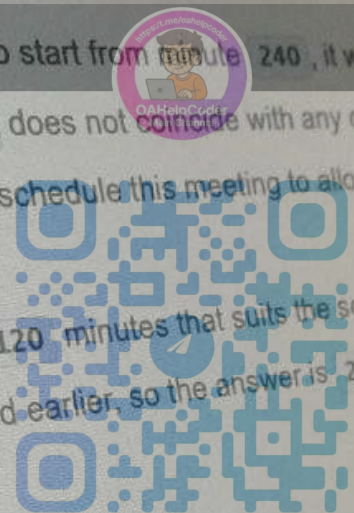
A meeting with a duration of 120 minutes that suits the schedule of all employees can't be scheduled earlier, so the answer is 240.

- For

```
schedules = [  
  [[480, 510]],  
  [[240, 330]],  
  [[375, 400]]  
]
```

and `length = 180`, the output should be `solution(schedules, length) =`

[MORE](#)



@OAHELPCODER

BACK TO QUESTIONS

DESC

HISTORY

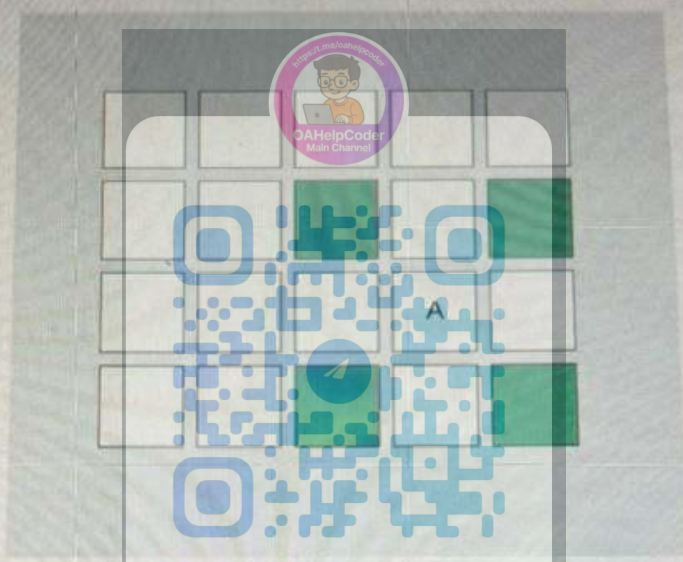
RULES

README

SETTINGS

Codewriting

Let's define the diagonal neighbors of a given cell as the surrounding cells, with precisely one corner touching a corner of the given cell. The picture below shows the diagonal neighbors for the A cell.



Given a board of cells containing a bubble of a specific color, your task is to emulate a bubble-popping game. In this game, the player can click a cell every turn to pop bubbles. After clicking the cell, the following happens:

main.cpp

```
1 vector<vector<int>>  
2  
3 }  
4
```

TESTS

MORE



Search

< BACK TO QUESTIONS

DESC



HISTORY



RULES



README



SETTINGS

- The bubble in the clicked cell and bubbles of the same color among its diagonal neighbors are "popped" and removed, resulting in empty cells.
- After bubbles are removed, the remaining bubbles in cells above the empty cells drop down to fill all empty cells.
- Nothing happens if the clicked cell is empty (it does not contain a bubble).

The input consists of two arrays:

- An initial board of cells `bubbles` - a multidimensional array of integers representing cells containing different colored bubbles;
- A set of player turns `operations` - 2-element integer arrays describing the coordinates (the row and column) of the cell that the player clicked on during each turn.

Return the state of the game board after all `operations` are processed. The output should be a multidimensional array of integers with the same size as `bubbles`, but replace the integers in all empty

main.cpp

```
1 vector<vector<int>>  
2  
3 }  
4
```

TESTS

: MORE



Search

Codewriting

Let's imagine objects located on the canvas at a certain moment in time. You are given an array of integer pairs `centers` representing the coordinates of those objects. Each object has a collision box - a square area around its center with a side equal to 2. Two objects are supposed to collide if their collision boxes have at least one common point. Calculate the number of object pairs that collide.

The example of the object located in coordinates `[1, 1]` can be represented as following:



MORE



Type here to search



Explanation:

- $|x[1] - x[0]| = 1$ and $|y[1] - y[0]| = 1$ both are ≤ 2 , so the first two objects collide.
- $|x[2] - x[1]| = 2$ and $|y[2] - y[1]| = 2$ both are ≤ 2 , so the last two objects collide.
- $|x[2] - x[0]| = 1$ which is ≤ 2 , but and $|y[2] - y[0]| = 3$ which is > 2 , so the first and the last object don't collide.

- For

```
centers = [  
    [1, 1],  
    [2, 2],  
    [0, 4],  
    [1, 1]  
]
```

the output should be

```
solution(centers) = 4
```

TESTS

Note

Object collision boxes intersect if the distance in each coordinate between the object centers does not exceed 2.

If x_1, y_1 are coordinates of one object and x_2, y_2 are coordinates of the second object, then the collision condition for them can be written in the form $|x[j] - x[i]| \leq 2$ and $|y[j] - y[i]| \leq 2$.

Example

- For

```
centers = [  
    [1, 1],  
    [2, 2],  
    [0, 4]  
]
```

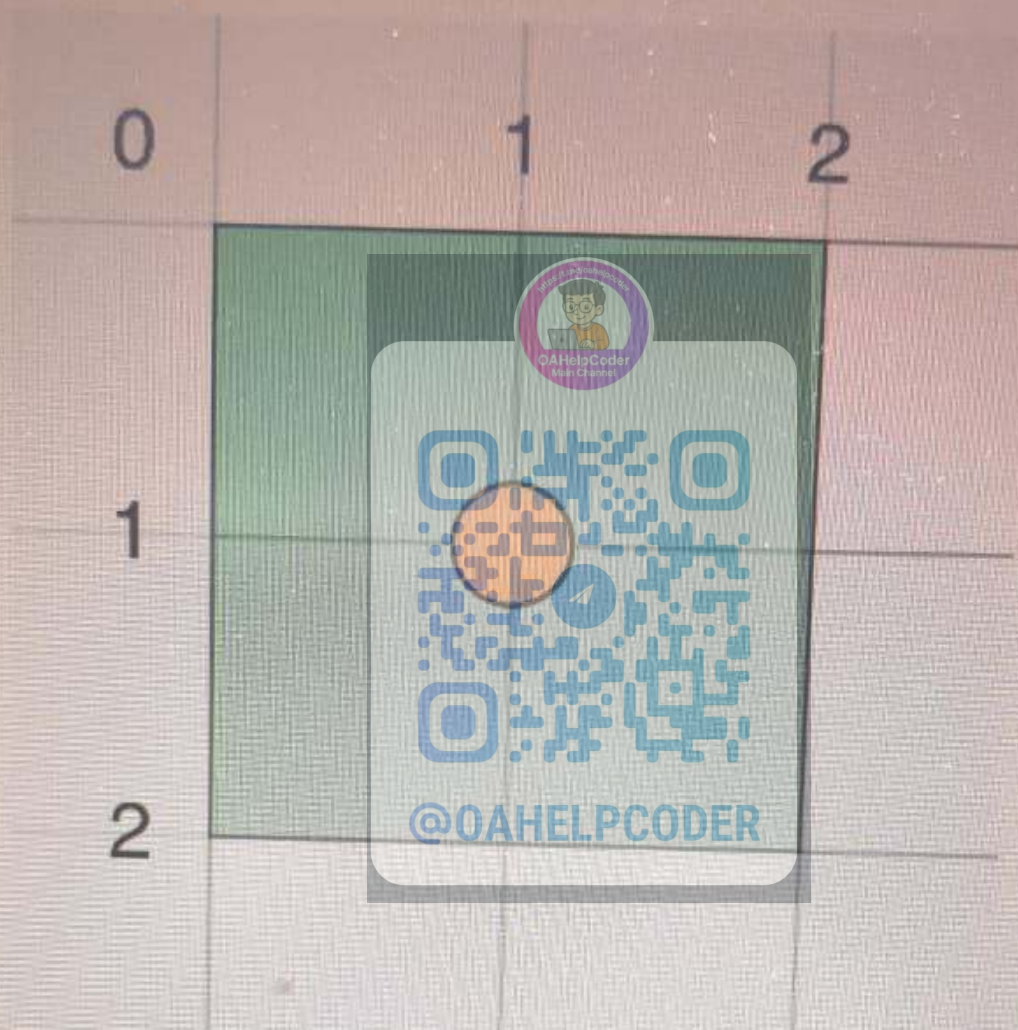
the output should be

```
solution(centers) 2
```

MORE

Type here to search

The example of the object located in coordinates `[1, 1]` can be represented as following:



Where the green square is the collision box.

Note

Object collision boxes intersect if the distance in each coordinate between the object

the output should be

`solution(centers) = 2`



Explanation:

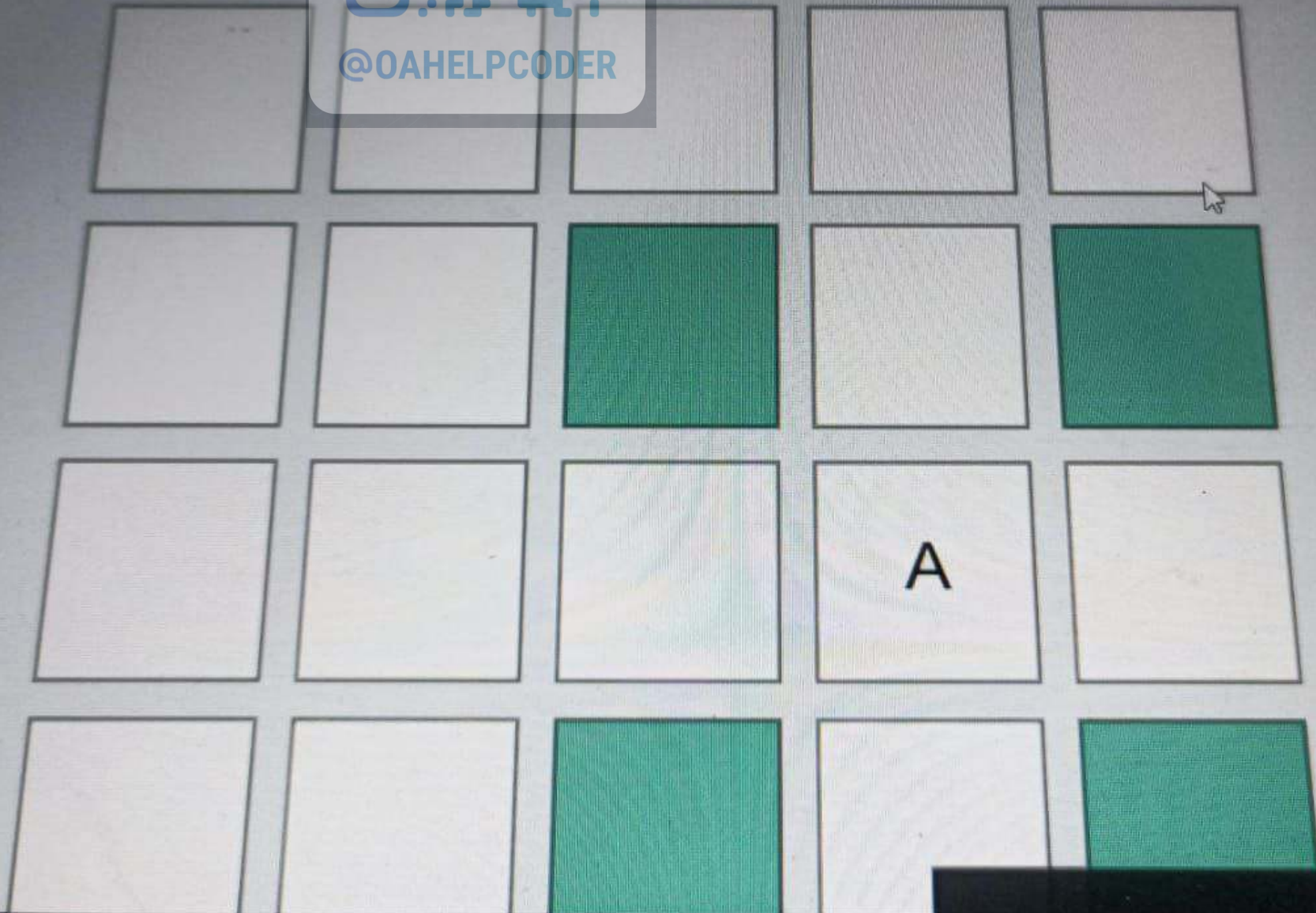
- $|x[1] - x[0]| = 1$ and $|y[1] - y[0]| = 1$ both are ≤ 2 , so the first two objects collide.
- $|x[2] - x[1]| = 2$ and $|y[2] - y[1]| = 2$ both are ≤ 2 , so the last two objects collide.
- $|x[2] - x[0]| = 1$ which is ≤ 2

TEST

Let's define the diagonal neighbors of a given cell as the surrounding cells, with precisely one corner touching a corner of the given cell. The picture below shows the diagonal neighbors for the A cell.



@OAHELPCODER



Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.array.integer bubbles

A multidimensional array of integers representing the initial game board of cells containing bubbles. Integers represent the colors of bubbles within cells, and it is guaranteed that all cells will have bubbles initially.

Guaranteed constraints:

$$1 \leq \text{bubbles.length} \leq 30 ,$$

$$1 \leq \text{bubbles}[0].\text{length} \leq 30 ,$$

$$1 \leq \text{bubbles}[i][j] \leq 900 .$$

- [input] array.array.integer operations

An array of 2-element arrays representing the player's moves - integers represent the coordinates of the row and column that the player has clicked on each turn.

Guaranteed constraints:

$$1 \leq \text{operations.length} \leq 200 ,$$

$$\text{operations}[i].\text{length} = 2 ,$$

$$0 \leq \text{operations}[i][0] \leq \text{bubbles.length} ,$$

Example

For

```
bubbles = [[1, 1, 1, 4, 3],  
            [4, 1, 2, 3, 3],  
            [1, 5, 1, 1, 2],  
            [4, 3, 2, 2, 4]]
```

@OAHELPCODER

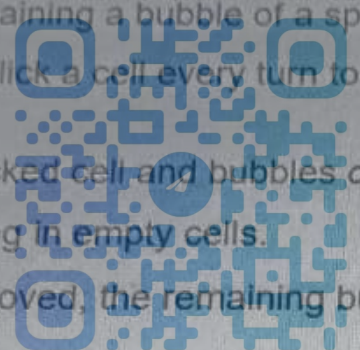
and

```
operations = [[1, 1],  
              [3, 3],  
              [2, 2],  
              [3, 0]]
```

the output should be

```
solution(bubbles, operations) = [[0, 0, 0, 0, 0],  
                                [0, 1, 0, 4, 3],  
                                [0, 5, 0, 3, 3],  
                                [4, 3, 2, 1, 4]]
```

▼ Expand to see the example video.



@OAHELPCODER

Given a board of cells containing a bubble of a specific color, your task is to emulate a bubble-popping game. In this game, the player can click a cell every turn to pop bubbles. After clicking the cell, the following happens:

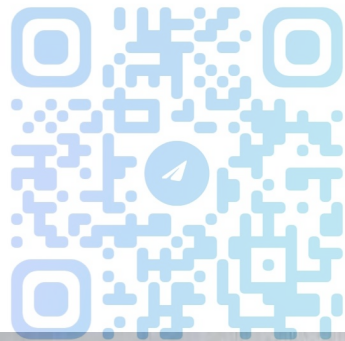
- The bubble in the clicked cell and bubbles of the same color among its diagonal neighbors are "popped" and removed, resulting in empty cells.
- After bubbles are removed, the remaining bubbles in cells above the empty cells drop down to fill all empty cells.
- Nothing happens if the clicked cell is empty (it does not contain a bubble).

The input consists of two arrays:

- An initial board of cells **bubbles** - a multidimensional array of integers representing cells containing different colored bubbles;
- A set of player turns **operations** - 2-element integer arrays describing the coordinates (the row and column) of the cell that the player clicked on during each turn.

Return the state of the game board after all **operations** are processed. The output should be a multidimensional array of integers with the same size as **bubbles**, but replace the integers in all empty cells (without bubbles) with **0**.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(bubbles.length^2 \cdot bubbles[0].length^2 \cdot operations.length)$ will fit within the execution time limit.



NGS An array of strings, where each string is either the existing **pattern** or **"not found"**.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloworld(string name) {
    cout << "This prints to the console when you Run Tests" << endl;
    return "Hello, " + name;
}
```


You are given a string of digits `panel` and an array of strings `codes`. Each string in the `codes` array consists of digits only and represents a code in the following format: "`<index><pattern>`", where both `index` and `pattern` should consist of at least one digit. Since there are several ways to split the code, let's consider them all in ascending order of `index` length and call them split-cases. For instance, for the `code = "1324"`, the split-cases are:

- split-case 1: `index = "1"` and `pattern = "324"` ;
- split-case 2: `index = "13"` and `pattern = "24"` ;
- split-case 3: `index = "132"` and `pattern = "4"` .

For each code in `codes` and for every split-case of this code, check whether a string `pattern` is present at the `index` position in the `panel` string. Return a string array consisting of results of these checks, where each element is either `pattern`, if this `pattern` is present in `panel`, or otherwise `"not found"`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{codes.length} \times \max(\text{codes}) \times \text{panel.length})$ will fit within the execution time limit.

Example

- For `panel = "2311453915"` and `codes = ["0211", "639"]`, the output should be `solution(panel, codes) = ["not found", "11", "not found", "39", "not found"]`.

Explanation:

Let's consider all possible codes:

- For `codes[0] = "021"` there are three split-cases:
 - `index` is `0` and `pattern` is `"211"`. Since `panel[0..2] = "231" ≠ pattern`, this is not a valid code. The answer for this split-case is `"not found"`.
 - `index` is `02` and `pattern` is `"11"`. `panel[2..3] = "11" = pattern`, so this is a valid code. The answer for this split-case is `"11"`.
 - `index` is `021` and `pattern` is `"1"`. As `panel[21]` doesn't exist, this is not a valid code. The answer for this split-case is `"not found"`.
 - For `codes[1] = "639"` there are two split-cases:
 - `index` is `6` and `pattern` is `"39"`. `panel[6..7] = "39" = pattern`, so this is a valid code. The answer for this split-case is `"39"`.
 - `index` is `63` and `pattern` is `"9"`. As `panel[63]` doesn't exist, this is not a valid code. The answer for this split-case is `"not found"`.
- The resulting array is `["not found", "11", "not found", "39", "not found"]`.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] string panel

• [execution time limit] 0.5 seconds (cpp)

• [memory limit] 1 GB

• [input] string panel

A string of digits.

Guaranteed constraints:

$1 \leq \text{panel.length} \leq 10^3$

• [input] array.string codes

An array of strings of digits.

Guaranteed constraints:

$1 \leq \text{codes.length} \leq 100$,

$2 \leq \text{codes}[i].\text{length} \leq 10$.

• [output] array.string

An array of strings, where each string is either the existing `pattern` or `"not found"`.

[C++] Syntax Tips

```
// Prints help message to the console  
// Returns a string
```

 Codewriting

You work for a busy office where employees schedule various events on their calendars. Your task is to analyze the data to determine the busiest month over the course of the years, based on the number of events.

Given a non-empty array of strings `dateList`, where each string represents a date in the format "YYYY-MM-DD", count the number of events for each month, ignoring the year. Return the month with the highest number of events as a two-digit numerical string (e.g., return "07" for July). In the event of a tie, return the latest month in the year.

Note that it is possible to have several events happening on the same date. We count those events as separate events.

Note: You are not expected to provide the



DESC



HISTORY



RULES



README



SETTINGS

MORE



Type here to search



- For `dateList = ["2025-05-10"]`, the output should be `solution(dateList)` = "05".

Explanation:

Here, there is only one date, and it is in May ("05"). Thus, May is the month with the highest number of events.

- For `dateList = ["2023-01-05", "2023-01-10", "2023-02-10", "2023-05-25", "2023-05-30"]`, the output should be `solution(dateList)` = "05".

Explanation:

January ("01") and May ("05") both have two events. Since there is a tie, we choose the latest month, which is May ("05").

Input/Output**TESTS**

A non-empty array of strings where each string represents a valid date in the format "YYYY-MM-DD".

Guaranteed constraints:

$1 \leq \text{dateList.length} \leq 1000$

- [output] string

The month with the highest number of events, formatted as a two-digit numerical string. If there is a tie, return the latest month in the year.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the conso
    return "Hello, " + name;
}
```

TESTS

Type here to search

Input/Output

- [execution time limit] 0.5 seconds (cpp)

- [memory limit] 1 GB

- [input] array<string> dateList

A non-empty array of strings where each string represents a valid date in the format "YYYY-MM-DD".

Guaranteed constraints.

$1 \leq \text{dateList.length} \leq 1000$

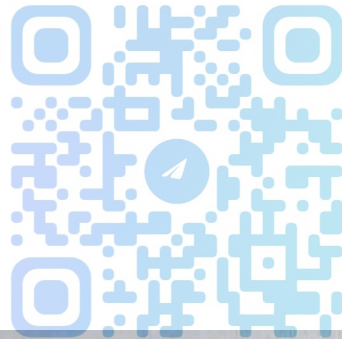
- [output] string

The month with the highest number of events, formatted as a two-digit numerical string. If there is a tie, return the latest month in the year.

[C++] Syntax Tips

TESTS

Type here to search



NGS An array of strings, where each string is either the existing **pattern** or **"not found"**.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloworld(string name) {
    cout << "This prints to the console when you Run Tests" << endl;
    return "Hello, " + name;
}
```


• [execution time limit] 0.5 seconds (cpp)

• [memory limit] 1 GB

• [input] string panel

A string of digits.

Guaranteed constraints:

$1 \leq \text{panel.length} \leq 10^3$

• [input] array.string codes

An array of strings of digits.

Guaranteed constraints:

$1 \leq \text{codes.length} \leq 100$,

$2 \leq \text{codes}[i].\text{length} \leq 10$.

• [output] array.string

An array of strings, where each string is either the existing `pattern` or `"not found"`.

[C++] Syntax Tips

```
// Prints help message to the console  
// Returns a string
```

You are given a string of digits `panel` and an array of strings `codes`. Each string in the `codes` array consists of digits only and represents a code in the following format: "`<index><pattern>`", where both `index` and `pattern` should consist of at least one digit. Since there are several ways to split the code, let's consider them all in ascending order of `index` length and call them split-cases. For instance, for the `code = "1324"`, the split-cases are:

- split-case 1: `index = "1"` and `pattern = "324"` ;
- split-case 2: `index = "13"` and `pattern = "24"` ;
- split-case 3: `index = "132"` and `pattern = "4"` .

For each code in `codes` and for every split-case of this code, check whether a string `pattern` is present at the `index` position in the `panel` string. Return a string array consisting of results of these checks, where each element is either `pattern`, if this `pattern` is present in `panel`, or otherwise `"not found"`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{codes.length} \times \max(\text{codes}) \times \text{panel.length})$ will fit within the execution time limit.

Example

- For `panel = "2311453915"` and `codes = ["0211", "639"]`, the output should be `solution(panel, codes) = ["not found", "11", "not found", "39", "not found"]`.

Explanation:

Let's consider all possible codes:

• For `codes[0] = "211"` there are three split-cases:

- `index` is 0 and `pattern` is "211". Since `panel[0..2] = "231" ≠ pattern`, this is not a valid code. The answer for this split-case is "not found".
- `index` is 02 and `pattern` is "11". `panel[2..3] = "11" = pattern`, so this is a valid code. The answer for this split-case is "11".
- `index` is 021 and `pattern` is "1". As `panel[21]` doesn't exist, this is not a valid code. The answer for this split-case is "not found".

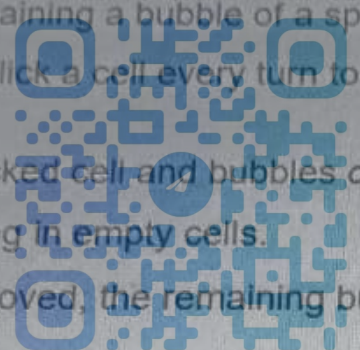
• For `codes[1] = "639"` there are two split-cases:

- `index` is 6 and `pattern` is "39". `panel[6..7] = "39" = pattern`, so this is a valid code. The answer for this split-case is "39".
- `index` is 63 and `pattern` is "9". As `panel[63]` doesn't exist, this is not a valid code. The answer for this split-case is "not found".

The resulting array is `["not found", "11", "not found", "39", "not found"]`.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] string panel



@OAHELPCODER

Given a board of cells containing a bubble of a specific color, your task is to emulate a bubble-popping game. In this game, the player can click a cell every turn to pop bubbles. After clicking the cell, the following happens:

- The bubble in the clicked cell and bubbles of the same color among its diagonal neighbors are "popped" and removed, resulting in empty cells.
- After bubbles are removed, the remaining bubbles in cells above the empty cells drop down to fill all empty cells.
- Nothing happens if the clicked cell is empty (it does not contain a bubble).

The input consists of two arrays:

- An initial board of cells **bubbles** - a multidimensional array of integers representing cells containing different colored bubbles;
- A set of player turns **operations** - 2-element integer arrays describing the coordinates (the row and column) of the cell that the player clicked on during each turn.

Return the state of the game board after all **operations** are processed. The output should be a multidimensional array of integers with the same size as **bubbles**, but replace the integers in all empty cells (without bubbles) with **0**.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(bubbles.length^2 \cdot bubbles[0].length^2 \cdot operations.length)$ will fit within the execution time limit.

Example

For

```
bubbles = [[1, 1, 1, 4, 3],  
            [4, 1, 2, 3, 3],  
            [1, 5, 1, 1, 2],  
            [4, 3, 2, 2, 4]]
```

@OAHELPCODER

and

```
operations = [[1, 1],  
              [3, 3],  
              [2, 2],  
              [3, 0]]
```

the output should be

```
solution(bubbles, operations) = [[0, 0, 0, 0, 0],  
                                 [0, 1, 0, 4, 3],  
                                 [0, 5, 0, 3, 3],  
                                 [4, 3, 2, 1, 4]]
```

▼ Expand to see the example video.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.array.integer bubbles

A multidimensional array of integers representing the initial game board of cells containing bubbles. Integers represent the colors of bubbles within cells, and it is guaranteed that all cells will have bubbles initially.

Guaranteed constraints:

$$1 \leq \text{bubbles.length} \leq 30 ,$$

$$1 \leq \text{bubbles}[0].\text{length} \leq 30 ,$$

$$1 \leq \text{bubbles}[i][j] \leq 900 .$$

- [input] array.array.integer operations

An array of 2-element arrays representing the player's moves - integers represent the coordinates of the row and column that the player has clicked on each turn.

Guaranteed constraints:

$$1 \leq \text{operations.length} \leq 200 ,$$

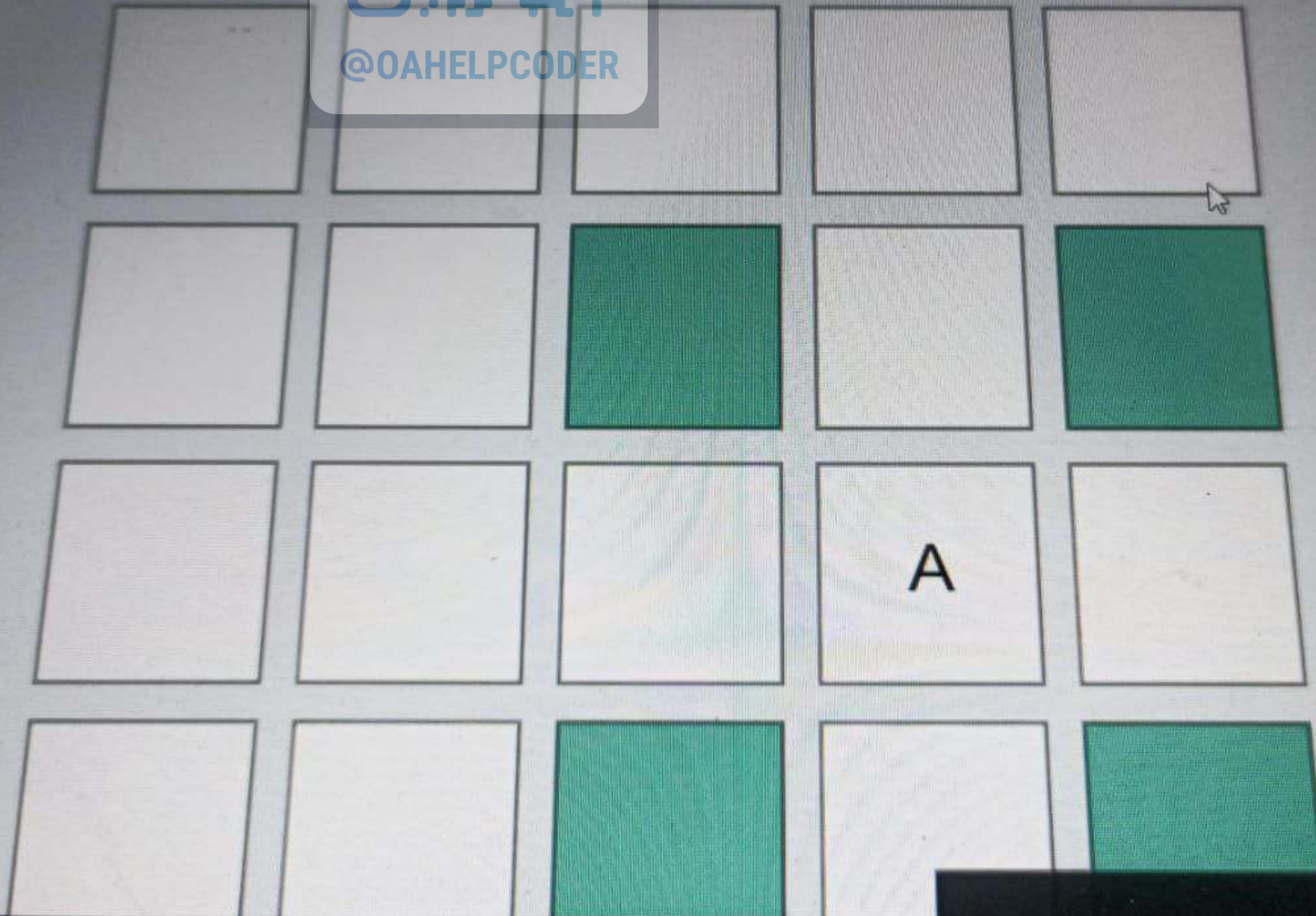
$$\text{operations}[i].\text{length} = 2 ,$$

$$0 \leq \text{operations}[i][0] \leq \text{bubbles.length} ,$$

Let's define the diagonal neighbors of a given cell as the surrounding cells, with precisely one corner touching a corner of the given cell. The picture below shows the diagonal neighbors for the A cell.



@OAHELPCODER



push command.

Also note that you are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\log s \cdot \text{length}^2)$ will fit within the execution time limit.

Example

- For

```
logs = [  
    "switch branch1",  
    "push file1",  
    "push file2",  
    "push file1",  
    "switch branch2",  
    "switch issue2",  
    "push file1",  
    "push file2",  
    "push file3"  
]
```

the output should be `solution(logs) = "issue2"`.

Explanation:

- Command 0: switch branch1
changes the current branch to

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.string logs

The list of logs executed across branches. It is guaranteed that commands follow the format described in the task description.

Guaranteed constraints:

$$2 \leq \text{logs.length} \leq 1000$$

- [output] string

The branch name with the largest number of files.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console whe
    return "Hello, " + name;
}
```

TESTS

- **Command 0:** `switch branch1` changes the current branch to `branch1`
- **Commands 1, 2:** The next two logs push `file1` and `file2` files, so `branch1` contains 2 files
- **Command 3:** Pushing `file1` in `branch1` doesn't change the content of the branch, as this file already exists there
- **Command 4:** `switch branch2` switches the current branch to `branch2`
- **Command 5:** `switch issue2` switches the current branch to `issue2`
- **Commands 6, 7, 8:** The last three logs push `file1`, `file2`, and `file3` to the `issue2` branch, so `issue2` contains 3 files

After all logs have been processed, here is the state of all the branches:

TESTS

- For `commands = "UDDUDD"`, the output should be `solution(commands) = "D"`.
 - After executing the first two commands, the robot will return to its starting position.
 - Similarly, after executing the third and fourth command, the robot will return to its starting position.
 - After executing the last two commands, the robot will stop 2 steps below its starting position, so the answer is `"D"`.
- For `commands = "DDUDDDDUUUU"`, the output should be `solution(commands) = ""`.
 - After executing all the commands, the robot will return to its starting position, so you should return an empty string.

Input/Output

- [execution time limit] 0.5 seconds (cpp)

❶ Codewriting

A robot responds to two commands, `u` or `d`, that tells it to move one step up (`u`) or one step down (`d`) on a vertical line.

You're given a string `commands` representing a series of `u` and `d` commands. After responding to all the commands in order, the robot will stop. Your task is to determine whether the robot will stop above or below its starting position:

- If the robot will stop above its starting position, return `"u"`.
- If the robot will stop at its starting position, return an empty string.
- If the robot will stop below its starting position, return `"d"`.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(commands.length^2)$ will fit within the execution time limit.

Example

Codewriting

For a rectangular matrix of integers, a *cross* is a figure formed by joining one row and one column. A cross is considered to be *regular* if all the elements in it are equal.

A cross is called *nearly regular* if all of its elements are equal except for, at times, the element in the intersection of the row and the column which form the cross.

You are given a rectangular matrix of integers `matrix`. Your task is to return the number of nearly regular crosses within `matrix`. Note that by definition the regular cross is also considered to be a nearly regular cross.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{matrix.length} \cdot$

$\text{matrix}[0].\text{length} \cdot (\text{matrix.length} + \text{matrix}[0].\text{length}))$ will fit within the execution time limit.

Example

After all logs have been processed, here is the state of all the branches:

- branch1 contains files file1 and file2
- branch2 is empty
- issue2 contains files file1, file2, and file3

As you can see, the branch with the largest number of files is issue2

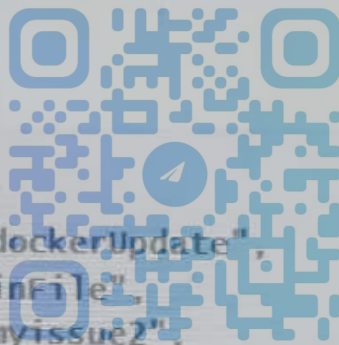
- For

```
logs = [  
    "switch dockerUpdate",  
    "push mainFile",  
    "switch myissue2",  
    "push configFile",  
    "push configFile",  
    "switch dockerUpdate",  
    "push configFile"  
]
```

the output should be `solution(logs) = "dockerUpdate"`.

Explanation:

After all logs have been processed, here is the



@OAHELPCODER

TEST

MORE

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] string commands

A string consisting of characters `D` and/or `U`.

Guaranteed constraints

$1 \leq \text{commands.length} \leq 1000$.

- [output] string

`"D"`, `"U"`, or `" "` depending on where the robot will stop relative to its starting position.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console whe
    return "Hello, " + name;
}
```

MORE

① Codewriting

You are developing a simple version control system. Your system can support branches, and you can switch between branches or update the files in a branch using the following actions:

- `switch <branch name>` – switches to the branch with given name
- `push <file name>` – pushes a file to the current branch. Note, that if the file was pushed to the current branch before, then the number of files should not be changed.

Your task is to process all the logs and return the branch name that contains the largest number of files at the end. It is guaranteed that there is no tie.

Notes:

- It is guaranteed that the first command is `switch`.
- It is guaranteed that there is at least one `push` command.

Also note that you are not expected to provide the